

Сценарий `linear_chain2.py` модуля `Drone_Swarm_Simulator_v2`: математическая модель, реализация и обоснование решений

Аннотация

В отчёте рассматривается Python-сценарий `scenarios/linear_chain2.py`, предназначенный для численного эксперимента в связке нескольких экземпляров ArduPilot SITL. Цель — сформировать **линейную цепочку** беспилотных аппаратов в горизонтальной плоскости: крайние агенты (якоря) удерживают концы отрезка AB , внутренние стабилизируют порядок на линии. Управление реализовано через переопределение радиоканалов (roll/pitch) в режиме позиционного удержания. Внутренний закон **гибридный**: геометрическая компонента (середина между соседями по проекции на AB и удержание на прямой) суммируется с уменьшенным вкладом распределённого закона из постановки «коридор» (`corridor_sweeping`): учёт ближайших соседей в четырёх полупространствах, нелинейность $\tanh$, виртуальный поперечный зазор при отсутствии наблюдения.

Ниже последовательно изложены постановка задачи, программная архитектура, системы координат, математические соотношения с пояснением каждой вновь вводимой величины, соответствие функций исходному коду и мотивы инженерных компромиссов.

Содержание

1	Введение и постановка задачи	3
1.1	Цель работы	3
1.2	Ограничения среды моделирования	3
1.3	Отличие от сценария <code>linear_chain.py</code>	3
2	Архитектура программного модуля	3
2.1	Общая схема	3
2.2	Инициализация агента	3
2.3	Обмен информацией о положении	4
2.4	Контур управления по ролям	4
2.5	Точка входа	4
3	Системы координат и отрезок цепи	4
3.1	Горизонтальная плоскость NED и поля позиции в коде	4
3.2	Общий кадр (common frame)	4
3.3	Построение концов отрезка A и B	4
3.4	Ортонормированный базис вдоль цепи	5
3.5	Одномерная координата вдоль отрезка	5
4	Математическая модель внутреннего контура	5
4.1	Множество наблюдаемых соседей и радиус видимости	6
4.2	Четыре полупространства: ближайшие расстояния	6
4.3	Вспомогательные функции нелинейности и виртуального зазора	6

4.4	Скаляры σ_a и σ_c	7
4.5	Геометрические ошибки e_{along} и e_{cross}	7
4.6	Комбинированный вектор в плоскости North–East	7
4.7	Мёртвые зоны и нейтраль RC	8
5	Преобразование s_{xy} в команды RC	8
5.1	Рыскание и поворот North–East $\rightarrow$ корпус	8
5.2	Демпфирование по скорости в корпусе	8
5.3	Пропорциональный закон, квантование PWM, минимальный шаг и slew . . .	8
5.4	Соответствие каналам автопилота	9
6	Контур якоря: слежение за точкой A или B	9
7	Соответствие фрагментов кода и математических блоков	9
8	Обоснование принятых решений	9
9	Параметры запуска и константы	10

1 Введение и постановка задачи

1.1 Цель работы

Требуется организовать совместное поведение роя из N беспилотных летательных аппаратов (в коде $N \geq 4$) в горизонтальной плоскости так, чтобы в установившемся режиме:

- два агента с **минимальным** и **максимальным** целочисленным идентификатором `id` выступали *якорями* и занимали (с допустимой погрешностью) заранее заданные концы отрезка AB длины L ;
- остальные агенты располагались на той же прямой между якорями с **близким к равномерным** упорядочением по положению вдоль отрезка.

Здесь N — число дронов в эксперименте; L — заданная длина отрезка в метрах (параметр `-segment-length`, по умолчанию совпадает с константой `SEGMENT\_LENGTH\_M` в файле).

1.2 Ограничения среды моделирования

Эксперимент выполняется в программной связке с ArduPilot SITL. На уровне сценария нет прямого задания вектора тяги в инерциальном базисе; вместо этого используется режим, эквивалентный удержанию позиции (`POS\_HOLD`), и низкоуровневое **переопределение каналов RC** (`MAV\_OVERRIDE`). Следовательно, закон управления должен быть сформулирован как *желаемое горизонтальное смещение* (в плоскости North–East), затем переведён в оси корпуса (вперёд–вправо) и дискретизован в приращения ширины импульса PWM по каналам крена и тангажа.

1.3 Отличие от сценария `linear_chain.py`

В версии 1 (`linear_chain.py`) внутренний дрон строит двумерную целевую точку на отрезке (середина между соседями по одномерной координате вдоль AB), сглаживает её и подаёт на PID-контур по roll/pitch. В `linear_chain2.py` **явная 2D-цель для внутренних не вводится**: формируется вектор $\mathbf{s}_{xy}$ в плоскости North–East как сумма геометрической и «коридорной» компонент, после чего к полученным сигналам в корпусе применяется пропорциональное преобразование в PWM с ограничениями, минимальным шагом и ограничением скорости изменения команды (slew rate). Такой подход проще согласовать с логикой `corridor_sweeping` и позволяет отдельно настраивать «жесткость» выравнивания по линии и «мягкость» нелинейного распределённого вклада.

2 Архитектура программного модуля

2.1 Общая схема

Модуль запускает несколько потоков POSIX: для каждого дрона — инициализация и основной контур поведения; дополнительно — поток обмена координатами между логическими контроллерами. Синхронизация старта после взлёта обеспечивается объектом `threading.Barrier`.

2.2 Инициализация агента

Функция `initialize\_drone\_parallel` для каждого экземпляра `DroneController` выполняет подключение по MAVLink, последовательность `INIT\_STEPS`: перевод в безопасный режим, армирование, взлёт на высоту `TAKEOFF\_ALT\_M`, переход в `POS\_HOLD`, установка нейтральных значений RC, запрос стримов позиции и ориентации с заданной частотой, запуск периодической посылки `keepalive` для удержания `override`. Таким образом, к началу эксперимента все агенты находятся в сопоставимом автопилотном режиме.

2.3 Обмен информацией о положении

Поток `coordinate\_exchange\_loop` с частотой f_{ex} (аргумент `-exchange-hz`, величина f_{ex} измеряется в герцах) опрашивает локальные оценки положения, переводит их в *общий кадр* (см. раздел 3) и записывает позиции остальных агентов в память каждого контроллера (`update\_other\_drone\_position`). Важно: это **не** модель бортовой связи; глобальная сводка позиций формируется в одном процессе симулятора. Для распределённого алгоритма на реальном рое потребовалась бы замена этого блока протоколом обмена измерениями.

2.4 Контур управления по ролям

После фиксации концов отрезка A и B (см. раздел 3.3) для якорей запускается `endpoint\_loop`, для внутренних — `internal\_chain\_anatoliy\_loop`. Период дискретизации контура управления $T_c = 1/f_c$, где f_c — `CONTROL\_HZ` (в текущей версии файла $f_c = 18$ Гц). Именно с шагом T_c вычисляются команды RC для соответствующего агента.

2.5 Точка входа

Функция `main()` разбирает аргументы командной строки, создаёт список конфигураций (идентификатор, UDP-порт и т. д.), поднимает потоки и по завершении корректно останавливает контроллеры. Параметры, влияющие на математику внутреннего закона: `-r-vis` (радиус видимости соседей R_{vis}), `-w-cross` (виртуальный поперечный зазор w_{cross}).

3 Системы координат и отрезок цепи

3.1 Горизонтальная плоскость NED и поля позиции в коде

В авиационной системе North–East–Down (NED) горизонтальные оси: **North** (на север) и **East** (на восток). В словаре позиции сценария компоненты обозначены x и y ; в дальнейшем отождествляем

$$x \equiv \text{смещение на север } (N), \quad y \equiv \text{смещение на восток } (E).$$

Компонента высоты z в NED направлена вниз; целевая высота взлёта задаётся отрицательным значением `-TAKEOFF\_ALT\_M` в тех же единицах, что ожидает автопилот.

3.2 Общий кадр (common frame)

У разных экземпляров SITL локальные координаты смещены относительно друг друга. Чтобы сравнивать положения агентов в одной системе, вводится поправка только по оси East (поле y):

$$y_{\text{common}} = y_{\text{local}} + \Delta y(\text{id}). \quad (1)$$

Здесь y_{local} — восточная координата, возвращаемая API контроллера для данного дрона; $\text{id} \in \{1, 2, \dots\}$ — целочисленный идентификатор агента в конфигурации сценария; $\Delta y(\text{id})$ — детерминированный сдвиг. В реализации $\Delta y(\text{id}) = (\text{id} - 1) \cdot 2 \text{ м}$ (функция `\_did\_offset\_y`). Смысл: развести траектории на визуализации и уменьшить ложное «наложение» траекторий при чтении локальных координат разных SITL. Северная координата x в common frame не сдвигается.

3.3 Построение концов отрезка A и B

Пусть после взлёта в common frame измерены положения левого и правого якоря; их восточные координаты обозначим y_{left} и y_{right} (это значения y после преобразования (1) для

якорей с минимальным и максимальным `id`). Задаётся средняя ордината

$$y_{\text{mid}} = \frac{y_{\text{left}} + y_{\text{right}}}{2}. \quad (2)$$

Величина y_{mid} — скаляр, фиксирующий **одну** горизонтальную прямую для геометрии цепи: оба конца отрезка лежат на одной «высоте» по East, даже если сами якоря в момент снимка слегка разошлись по y из-за шума оценки или задержек взлёта.

Длина отрезка $L > 0$ задаётся параметром запуска. Точки A и B в горизонтальной плоскости (в координатах North–East относительно принятого начала common frame) задаются как

$$A = (N_A, E_A) = \left(-\frac{L}{2}, y_{\text{mid}}\right), \quad B = (N_B, E_B) = \left(+\frac{L}{2}, y_{\text{mid}}\right). \quad (3)$$

Таким образом, N_A, N_B — северные координаты концов; $E_A = E_B = y_{\text{mid}}$ — восточные. В коде тем же значениям соответствуют поля `x` и `y` словарей `endpoint\_a\_common` и `endpoint\_b\_common`.

3.4 Ортонормированный базис вдоль цепи

Введём единичный вектор направления от A к B в плоскости North–East:

$$\mathbf{u} = \frac{(N_B - N_A, E_B - E_A)^\top}{\sqrt{(N_B - N_A)^2 + (E_B - E_A)^2}} = (u_N, u_E)^\top. \quad (4)$$

Компоненты u_N и u_E — направляющие косинусы оси цепи. Левый ортогональный вектор в той же плоскости (поворот на $+90^\circ$ против часовой стрелки в плане North–East, если смотреть сверху вниз в стандартной ориентации NED):

$$\mathbf{n} = (-u_E, u_N)^\top = (n_N, n_E)^\top. \quad (5)$$

Вектор $\mathbf{u}$ направлен вдоль цепи от левого якоря к правому; $\mathbf{n}$ задаёт ось «влево» от направления полёта вдоль цепи. Реализация: `\_unit\_vector\_xy`, `\_normal\_left\_xy`.

3.5 Одномерная координата вдоль отрезка

Для произвольной точки p с горизонтальными координатами (N_p, E_p) определим скалярную координату вдоль отрезка (проекцию на ось от точки A):

$$s(p) = (N_p - N_A) u_N + (E_p - E_A) u_E. \quad (6)$$

Величина $s(p)$ измеряется в метрах; она монотонно возрастает при движении от A к B вдоль прямой. Эта координата используется для упорядочивания агентов «вдоль цепи» независимо от наклона прямой в плоскости (при невырожденном $A \neq B$).

4 Математическая модель внутреннего контура

Внутренний агент (не якорь) в каждый момент дискретного времени с шагом T_c выполняет вычисления ниже. Обозначим через p_{me} вектор его положения в common frame в горизонтальной плоскости $(N_{\text{me}}, E_{\text{me}})$, через $\mathcal{P}$ — множество идентификаторов агентов, чьи позиции доступны (своя и чужие после обмена).

4.1 Множество наблюдаемых соседей и радиус видимости

Пусть $R_{\text{vis}} > 0$ — радиус видимости (параметр `R\VIS\М` или `-r-vis`). Для каждого другого агента $j \in \mathcal{P}$, $j \neq \text{my_id}$, вычисляется евклидово расстояние в плоскости

$$d_j = \|p_{j,xy} - p_{\text{me},xy}\|_2. \quad (7)$$

Здесь $p_{j,xy}$ — горизонтальные координаты агента j ; `my_id` — идентификатор рассматриваемого дрона. Агент j учитывается, если $d_j \leq R_{\text{vis}}$ и d_j не ниже малый порог (в коде исключение нулевого вектора). Для каждого учтённого j вводится вектор относительного положения в плоскости

$$\Delta_j = p_{j,xy} - p_{\text{me},xy} = (\Delta_{j,N}, \Delta_{j,E})^\top. \quad (8)$$

Проекции Δ_j на базис цепи $(\mathbf{u}, \mathbf{n})$:

$$a_j = \Delta_j \cdot \mathbf{u} = \Delta_{j,N}u_N + \Delta_{j,E}u_E, \quad c_j = \Delta_j \cdot \mathbf{n} = \Delta_{j,N}n_N + \Delta_{j,E}n_E. \quad (9)$$

Скаляр a_j — смещение соседа **вдоль** цепи относительно рассматриваемого дрона (положительное — в сторону $\mathbf{u}$); c_j — смещение **поперёк** (положительное — в сторону $\mathbf{n}$). Если число таких соседей меньше двух, команда RC сбрасывается в нейтраль (недостаточно информации для устойчивого закона).

4.2 Четыре полупространства: ближайшие расстояния

По совокупности пар (a_j, c_j) находятся минимальные положительные смещения в каждом из четырёх квадрантов осей (a, c) . Вводим:

$$\begin{aligned} d_{a+} &= \min\{a_j : a_j \geq 0, j \text{ учтён}\}, & d_{a-} &= \min\{|a_j| : a_j < 0, j \text{ учтён}\}, \\ d_{c+} &= \min\{c_j : c_j \geq 0, j \text{ учтён}\}, & d_{c-} &= \min\{|c_j| : c_j < 0, j \text{ учтён}\}. \end{aligned}$$

Если множество в определении пусто, соответствующее расстояние считается бесконечным, а булев флаг наличия соседа в данном полупространстве становится ложным. Таким образом, $d_{a+}, d_{a-}, d_{c+}, d_{c-}$ — неотрицательные расстояния до ближайшего соседа впереди/сзади вдоль цепи и влево/вправо поперёк; флаги $f_{a+}, f_{a-}, f_{c+}, f_{c-}$ фиксируют, было ли направление «занято» наблюдением. Реализация: `\_nearest\_along\_cross`.

4.3 Вспомогательные функции нелинейности и виртуального зазора

Определим ограниченную нелинейность

$$\xi(x) = \tanh(x), \quad x \in \mathbb{R}. \quad (10)$$

Функция ξ сжимает вклад больших расстояний и ограничивает выход интервалом $(-1, 1)$, что сглаживает реакцию при тесном сближении.

Для произвольного неотрицательного d , константы зазора $w \geq 0$ и булева флага f вводится

$$g(d, w, f) = \begin{cases} d, & f = \text{истина}, \\ w, & f = \text{ложь}. \end{cases} \quad (11)$$

То есть при отсутствии соседа в заданном полупространстве вместо «бесконечного» расстояния подставляется конечный зазор w . В коде для оси вдоль при $f = \text{ложь}$ берётся $w = 0$; для поперечной оси — $w = w_{\text{cross}}$, где $w_{\text{cross}} > 0$ (`W\_CROSS\М` или `-w-cross`). Смысл w_{cross} — имитация **виртуальной стенки** коридора с той стороны, где нет наблюдаемого соседа, чтобы скалярный сигнал не вырождался.

4.4 Скаляры σ_a и σ_c

В полной динамической постановке «коридора» в сигнал входят проекции скорости агента на оси $(\mathbf{u}, \mathbf{n})$:

$$v_{\text{along}} = v_N u_N + v_E u_E, \quad v_{\text{cross}} = v_N n_N + v_E n_E, \quad (12)$$

где (v_N, v_E) — горизонтальная скорость в NED. В рассматриваемом файле для устойчивости в SITL принято

$$v_{\text{along}} = 0, \quad v_{\text{cross}} = 0. \quad (13)$$

Тогда

$$\sigma_a = v_{\text{along}} - \xi(g(d_{a+}, 0, f_{a+})) + \xi(g(d_{a-}, 0, f_{a-})), \quad (14)$$

$$\sigma_c = v_{\text{cross}} - \xi(g(d_{c+}, w_{\text{cross}}, f_{c+})) + \xi(g(d_{c-}, w_{\text{cross}}, f_{c-})). \quad (15)$$

Величины σ_a и σ_c — скалярные сигналы «распределённого» закона в базисе цепи: они сравнивают влияние ближайших соседей с противоположных сторон по каждой оси с учётом насыщения. При $v = 0$ остаётся статическая нелинейная функция от положений соседей. Реализация: `\_sigmas\_segment\_frame`.

4.5 Геометрические ошибки e_{along} и e_{cross}

Все агенты с известными позициями упорядочиваются по возрастанию пары $(s(p_i), \text{id}_i)$, где s определено в (6). Пусть у внутреннего агента есть левый и правый сосед в этом порядке с координатами s_L и s_R (проекции (6) для соседей). Тогда ошибка **равномерности вдоль цепи**:

$$e_{\text{along}} = \frac{s_L + s_R}{2} - s_{\text{me}}, \quad (16)$$

где $s_{\text{me}} = s(p_{\text{me}})$. Если соседа слева или справа нет (агент оказался на краю упорядоченного списка при данном снимке), полагают $e_{\text{along}} = 0$ в этой реализации.

Ошибка **удержания на прямой AB** (signed cross-track distance):

$$e_{\text{cross}} = (N_{\text{me}} - N_A) n_N + (E_{\text{me}} - E_A) n_E. \quad (17)$$

Положительное e_{cross} означает смещение в сторону $\mathbf{n}$ относительно прямой через A .

Обе ошибки затем ограничиваются по модулю константами $E_{\text{along}}^{\text{max}}$ и $E_{\text{cross}}^{\text{max}}$ (`SPACING\_E\_ALONG\_CLAMP`, `SPACING\_E\_CROSS\_CLAMP`), чтобы ограничить максимальное «натяжение» геометрической компоненты при выбросах измерений.

4.6 Комбинированный вектор в плоскости North–East

Вводятся положительные коэффициенты k_{along} и k_{cross} (`SPACING\_ALONG\_K`, `SPACING\_CROSS\_K`). Геометрическая составляющая:

$$\mathbf{g}_{xy} = k_{\text{along}} e_{\text{along}} \mathbf{u} - k_{\text{cross}} e_{\text{cross}} \mathbf{n} = (g_N, g_E)^\top. \quad (18)$$

Знак «−» перед поперечной ошибкой соответствует притягиванию к прямой в сторону, противоположную $\mathbf{n}$ при положительном e_{cross} (согласовано с реализацией в коде).

Вклад закона «коридора»:

$$\mathbf{a}_{xy} = \sigma_a \mathbf{u} + \sigma_c \mathbf{n} = (a_N, a_E)^\top. \quad (19)$$

Вес $w_A \in [0, 1]$ (`ANATOLIY\_COMB\_WEIGHT`) задаёт долю «анатолиевской» компоненты. Итоговый вектор желаемого горизонтального усилия в NED-плоскости:

$$\mathbf{c}_{xy} = \mathbf{g}_{xy} + w_A \mathbf{a}_{xy} = (c_N, c_E)^\top. \quad (20)$$

4.7 Мёртвые зоны и нейтраль RC

Пусть ε_σ — порог `SIGMA\_DEADBAND`, ε_s — порог `SPACING\_E\_DEADBAND\_M`. Вводится булева величина «есть геометрическое натяжение»:

$$\text{pull}_{\text{geom}} = (|e_{\text{along}}| > \varepsilon_s) \vee (|e_{\text{cross}}| > \varepsilon_s). \quad (21)$$

Если $\text{pull}_{\text{geom}}$ ложно и одновременно $|\sigma_a| < \varepsilon_\sigma$, $|\sigma_c| < \varepsilon_\sigma$, на агент подаётся нейтральный RC (команда «не вмешиваться»).

5 Преобразование c_{xy} в команды RC

5.1 Рыскание и поворот North–East $\rightarrow$ корпус

Пусть ψ — угол рыскания (yaw) в радианах, полученный из потока attitude; в коде к нему применяется множитель $s_y \in \{-1, +1\}$ (`YAW\_SIGN`) для согласования знака с конкретной сборкой SITL. Компоненты вектора c_{xy} в North–East обозначим c_N, c_E (совпадают с c_N, c_E из (20)). Желаемые «вперёд» и «вправо» в системе, связанной с корпусом (ось вперёд — ось носа, вправо — к правому крылу), до демпфирования:

$$\begin{pmatrix} f_0 \\ r_0 \end{pmatrix} = \begin{pmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{pmatrix} \begin{pmatrix} c_N \\ c_E \end{pmatrix}. \quad (22)$$

Здесь f_0 — желаемое усилие «вперёд»; r_0 — «вправо». Реализация: `\_ned\_to\_body\_forward\_right`.

5.2 Демпфирование по скорости в корпусе

Горизонтальная скорость (v_N, v_E) из стрима скоростей переводится в корпусные (v_f, v_r) той же матрицей (22). Вводятся коэффициент $k_d > 0$ (`INTERNAL\_BODY\_V\_DAMP`) и предел насыщения $D_{\max} > 0$ (`INTERNAL\_BODY\_V\_DAMP\_CLAMP`). Оператор sat_D — ограничение по модулю: $\text{sat}_D(z) = \text{sign}(z) \min(|z|, D)$. Тогда

$$f = f_0 - \text{sat}_{D_{\max}}(k_d v_f), \quad r = r_0 - \text{sat}_{D_{\max}}(k_d v_r). \quad (23)$$

Смысл: если аппарат уже движется в корпусных осях в ту же сторону, команду уменьшают, снижая перерегулирование.

5.3 Пропорциональный закон, квантование PWM, минимальный шаг и slew

Пусть $k_p > 0$ — `INTERNAL\_RC\_KP`, M_{PWM} — `INTERNAL\_RC\_MAX\_PWM`, $\delta_{\min}$ — `INTERNAL\_RC\_MIN\_STEP\_PWM`, $S_{\max}$ — `INTERNAL\_RC\_SLEW\_PWM\_PER\_CYCLE`. Для канала pitch целевое приращение относительно нейтралы строится из f ; для roll используется $k_{p,r} = k_p \cdot m_r$, где m_r — `INTERNAL\_RC\_KP\_ROLL\_MULT` (усилие крена относительно тангажа).

На каждом шаге сначала вычисляют «сырое» целое приращение $\text{round}(k_p f)$ (и аналогично для r с $k_{p,r}$), ограничивают по модулю M_{PWM} . Если ошибка по оси ненулевая, а округление дало ноль, подставляют $\pm \delta_{\min}$ по знаку ошибки — **минимальный шаг** (bang-bang по малым отклонениям), чтобы избежать залипания на нейтралы из-за квантования. Затем к предыдущему выходу применяют ограничение приращения за один период T_c не более $S_{\max}$ (slew rate). Обозначим результаты Δ_{pitch} и Δ_{roll} — целые приращения PWM после slew. Реализации: `\_pwm\_axis`, `\_slew\_int`.

5.4 Соответствие каналам автопилота

Пусть U_0 — нейтральное значение PWM (`RC\_NEUTRAL`). Тогда

$$U_{\text{pitch}} = U_0 - \Delta_{\text{pitch}}, \quad U_{\text{roll}} = U_0 + \Delta_{\text{roll}}, \quad (24)$$

с последующим ограничением в допустимый диапазон приёмника. При несовпадении карты каналов у пользователя предусмотрен флаг `RC\_OVERRIDE\_SWAP\_ROLL\_PITCH`.

6 Контур якоря: слежение за точкой A или B

Якорь получает целевую точку $T \in \{A, B\}$ с координатами (N_T, E_T) . Ошибка положения в North-East:

$$e_N = N_T - N_{\text{me}}, \quad e_E = E_T - E_{\text{me}}. \quad (25)$$

К компонентам применяется мёртвая зона по модулю ε_x (`ERROR\_DEADBAND\_M`): малые e_N, e_E обнуляются. Далее ошибка переводится в корпус (e_f, e_r) по формуле (22) (с тем же ψ). Два дискретных PID-регулятора (в текущей конфигурации $K_i = K_d = 0$, то есть пропорциональные каналы с разными коэффициентами) выдают $u_{\text{pitch}} = \text{PID}_p(e_f)$ и $u_{\text{roll}} = \text{PID}_r(e_r)$ с ограничением выхода `ENDPOINT\_OUTPUT\_LIMIT`. Маппинг на PWM совпадает по структуре с (22)-каналами для внутренних, но коэффициенты усиления другие (`ENDPOINT\_X\_KP` связан с продольной ошибкой в корпусе через `pitch`, `ENDPOINT\_Y\_KP` — с поперечной через `roll`). После попадания в круг радиуса ρ_{end} (`ENDPOINT\_REACHED\_THRESH\_M`) якорь переключается на удержание нейтральным RC.

Реализация: `endpoint\_loop`, `\_move\_towards\_common\_with\_pid`.

7 Соответствие фрагментов кода и математических блоков

Функция / блок	Назначение
<code>_segment_endpoints_common_frame_anchors</code>	Вывод точек A, B по (2)–(3).
<code>_unit_vector_xy</code> , <code>_normal_left_xy</code>	Базис $\mathbf{u}, \mathbf{n}$ по (4)–(5).
<code>_projection_s</code>	Скаляр $s(p)$ по (6).
<code>_nearest_along_cross</code>	Расстояния $d_{a\pm}, d_{c\pm}$ и флаги f .
<code>_xi, _g</code> , <code>_sigmas_segment_frame</code>	Нелинейность ξ , зазор g , сигналы σ_a, σ_c по (14).
<code>_spacing_errors_projection</code>	Комбини $e_{\text{along}}, e_{\text{cross}}$ по (16)–(17).
<code>internal_chain_anatoliy</code>	Сбор s_{xy} , поворот, демпфирование, PWM.
<code>_pwm_axis</code> , <code>_slew_int</code>	Пропорциональное квантование, минимальный шаг, slew .
<code>_move_towards_common_with_pid</code>	Автопилот PID в корпусе.
<code>coordinate_exchange_loop</code>	Рассылка позиций в common frame с частотой f_{ex} .

8 Обоснование принятых решений

Базис $(\mathbf{u}, \mathbf{n})$ привязан к отрезку AB .

Распределённый закон коридора изначально формулируется в осях «вдоль границы / поперёк». Использование $\mathbf{u}$ и $\mathbf{n}$ делает описание **инвариантным к наклону** цепи в плоскости и согласует геометрию с законом Anatoliy без пересчёта в мировые оси симулятора.

Общая ордината y_{mid} для A и B .

Раздельное использование y_{left} и y_{right} при реальных шумах и задержках взлёта часто даёт наклонную целевую прямую: якоря «тянут» внутренних к разным поперечным уровням, что визуально проявляется как излом цепи. Среднее (2) фиксирует одну прямую для геометрической компоненты.

Сумма $\mathbf{g}_{xy} + w_A \mathbf{a}_{xy}$ с малым w_A .

Чистый «коридорный» закон оптимизирован для движения вдоль границы с препятствиями и может конкурировать с требованием лечь на одну прямую между якорями. Доминирование $\mathbf{g}_{xy}$ обеспечивает выравнивание по проекции s и удержание на AB ; малый вес w_A добавляет нелинейную распределённую поправку без разрушения формы цепи.

Нелинейность $\tanh$.

Ограничивает чувствительность при малых d и предотвращает неограниченный рост сигнала при сближении — стандартный приём мягкого взаимодействия в многоагентных законах.

Виртуальный зазор w_{cross} .

При отсутствии наблюдаемого соседа с одной стороны поперёк подстановка конечного w_{cross} в g сохраняет ненулевую производную по положению и имитирует наличие границы коридора.

Обнуление $v_{\text{along}}, v_{\text{cross}}$ в (14).

Оценки скорости из MAVLink в SITL могут быть зашумлены или несогласованы с ожидаемым земным NED; включение скорости приводило бы к дрейфу σ . Принят статический компромисс: закон опирается на положения соседей.

RC override и дискретный контур T_c .

Единый интерфейс с автопилотом в режиме удержания; минимальный шаг PWM и slew ограничивают рывки и перегрузку контура на частоте f_c .

Поворот в корпус перед формированием PWM.

Команды roll/pitch действуют в плоскости корпуса; при $\psi \neq 0$ прямое использование (e_N, e_E) или (c_N, c_E) без поворота (22) исказило бы направление усилия.

YAW_SIGN и обмен roll/pitch.

Устранение расхождений знака yaw между прошивкой SITL и принятой моделью NED→body; учёт пользовательской карты каналов RC.

9 Параметры запуска и константы

Аргументы командной строки: `-drones` (N), `-duration` (длительность эксперимента, 0 — без ограничения), `-segment-length` (L), `-exchange-hz` (f_{ex}), `-r-vis` (R_{vis}), `-w-cross` (w_{cross}). Полный перечень именованных констант и значений по умолчанию содержится в начале файла `linear_chain2.py`; сжатое описание на русском — в `linear_chain2\_control.md`.

Заключение

В работе описан сценарий `linear_chain2.py`: многоагентное формирование линейной цепи в симуляторе с разделением ролей (якоря и внутренние агенты), вводом общего кадра координат, построением отрезка AB и локального базиса $(\mathbf{u}, \mathbf{n})$. Для внутренних агентов получена явная форма вектора $\mathbf{c}_{xy}$ в плоскости North–East как суммы геометрической компоненты (18) и взвешенной «коридорной» (19), далее показан переход к командам RC через поворот в корпус (22), демпфирование (23) и дискретное ограниченное преобразование в PWM. Приведены обоснования ключевых компромиссов (средняя ордината концов, гибридный закон, $v = 0$ в σ , квантование и slew). Для проверки соответствия формул кодовой базе рекомендуется сопоставлять номера уравнений с указанными функциями при каждом изменении исходного файла.